

MiniProject2_WebScraping

Cynthia Budiyanto and Willemijn Leiner

```
library(rvest)
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.1.4      v readr      2.1.6
v forcats    1.0.1      v stringr    1.6.0
v ggplot2     4.0.1      v tibble     3.3.1
v lubridate  1.9.4      v tidyr      1.3.2
v purrr      1.2.1
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter()          masks stats::filter()
x readr::guess_encoding() masks rvest::guess_encoding()
x dplyr::lag()             masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(stringr)
library(polite)
library(htmltools)
library(purrr)
```

```
# Defining the url to be its own value to make it easier to work with
url <- "https://link.springer.com/search?facet-journal-id=12968&query=machine+learning&content-type=article"

# Parse the HTML content of the page
page <- read_html(url)

# Extracting article titles using the CSS selector
titles <- page |>
  html_elements("a.app-card-open__link") |>
  html_text()
```

```

titles

# Extracting relative URLs for each article
links <- page |>
  html_elements("a.app-card-open__link") |>
  html_attr("href")
links <- paste0("https://link.springer.com", links)

# Extracting publication date
dates <- page |>
  html_elements(".app-card-open__meta") |>
  html_text()

# Cleaning the date strings
dates_clean <- dates |>
  str_squish()
dates_clean <- dates_clean |>
  str_remove("Open access")
dates_clean <- dates_clean |>
  str_squish()
dates_clean

# Combining the extracted data into a single tibble for inspection
articles <- tibble(
  title = titles,
  date = dates_clean,
  link = links
)

view(articles)

```

FUNCTION 1: SCRAPING INDIVIDUAL ARTICLE LINKS

```

get_link_from_search <- function(page, css_code) {                                # defining the function
  link <- page |>                                                                    # piping the object 'page'
    html_elements(css_code) |>                                                    # find all html elements
    html_attr("href")                                                            # extracts the value of href
}

scrape_list_links1 <- function(url) {                                              # defining the function

```

```

Sys.sleep(3)
session <- bow(url, force = TRUE)
page <- scrape(session)
list_links <- get_link_from_search(page, ".app-card-open_link")
complete_link <- paste0("https://link.springer.com", list_links)
list(complete_link)
}

```

polite scraping
creates a browser session
scrape the given URL
extract the links
stitching the extra links
wraps it into a list

FUNCTION 2: USING LINKS TO GET INDIVIDUAL ARTICLE INFO

```

get_info_from_page <- function(page, css_code) {
  result <- page |>
    html_elements(css_code) |>
    html_text(trim = TRUE)
  if (length(result) == 0)
    return(NA_character_)
  result[1]
}

scrape_individual_article <- function(x) {
  Sys.sleep(3)
  session <- bow(x, force = TRUE)
  page <- scrape(session)
  jcmr_title <- get_info_from_page(page,
    ".c-article-title")
  jcmr_vol <- get_info_from_page(page,
    ".c-article-identifiers--cite-list .c-article-identifiers__item:1")
  jcmr_abstract <- get_info_from_page(page,
    "#Abs1-content p:nth-child(2)")
  jcmr_method <- get_info_from_page(page,
    "#Abs1-content p:nth-child(4)")
  jcmr_author <- get_info_from_page(page,
    "#main-content header")
  jcmr_type <- get_info_from_page(page,
    ".c-article-title+ .c-article-identifiers .c-article-identifiers__item:1")
  jcmr_date <- get_info_from_page(page,
    ".c-article-identifiers__item time")
  tibble(date = jcmr_date,
    title = jcmr_title,
    volume = jcmr_vol,

```

defining the function
piping the object
find all of the HTML elements
retrieve text content
if length of the string is 0
then return the value NA
take the first match

defining the function
polite scraping
creates a browser session
scrape the given URL
scrape the "title"
scrape the "volume"
scrape the "abstract"
scrape the "method"
scrape the "author"
scrape the "article type"
scrape the "published date"
create a tibble containing the data

```

    authors = jcmr_author,
    type = jcmr_type,
    abstract = jcmr_abstract,
    method = jcmr_method)
}

```

KEYWORD 1: “Machine Learning” 3 PAGES OF WEBSITE

```

# Scraping links from PAGE 1 - MACHINE LEARNING
pg1_ml <- scrape_list_links1("https://link.springer.com/search?query=machine+learning&content-type=article")

# Scraping links from PAGE 2 - MACHINE LEARNING
pg2_ml <- scrape_list_links1("https://link.springer.com/search?query=machine+learning&content-type=article")

# Scraping links from PAGE 3 - MACHINE LEARNING
pg3_ml <- scrape_list_links1("https://link.springer.com/search?query=machine+learning&content-type=article")

# Combining two pages into one list of links (55 links total)
ml <- list(pg1_ml, pg2_ml, pg3_ml)
clean_ml <- unlist(ml)

# Scraping information from each individual article
details_kw1 <- purrr::map(clean_ml, scrape_individual_article) # Using our pre-made function
bind_rows(details_kw1)

# Turning the binded rows into a tibble
details_kw1 <- bind_rows(details_kw1)

```

KEYWORD 2: “Deep Learning” 3 PAGES OF WEBSITE

```

# Scraping links from PAGE 1 - DEEP LEARNING
pg1_dl <- scrape_list_links1("https://link.springer.com/search?query=deep+learning&content-type=article")

# Scraping links from PAGE 2 - DEEP LEARNING
pg2_dl <- scrape_list_links1("https://link.springer.com/search?query=deep+learning&content-type=article")

# Scraping links from PAGE 3 - DEEP LEARNING
pg3_dl <- scrape_list_links1("https://link.springer.com/search?query=deep+learning&content-type=article")

# Combining two pages into one list of links (54 links total)

```

```

dl <- list(pg1_dl, pg2_dl, pg3_dl)
clean_dl <- unlist(dl)

# Scraping information from each individual article
details_kw2 <- purrr::map(clean_dl, scrape_individual_article)      # Using our pre-made f
bind_rows(details_kw2)

# Turning the binded rows into a tibble
details_kw2 <- bind_rows(details_kw2)

```

KEYWORD 3: “Convolution” 2 PAGES OF WEBSITE

```

# Scraping links from PAGE 1 - CONVOLUTION
pg1_con <- scrape_list_links1("https://link.springer.com/search?query=convolution&content-type=article")

# Scraping links from PAGE 2 - CONVOLUTION
pg2_con <- scrape_list_links1("https://link.springer.com/search?query=convolution&content-type=article")

# Combining two pages into one list of links (39 links total)
con <- list(pg1_con, pg2_con)
clean_con <- unlist(con)

# Scraping information from each individual article
details_kw3 <- purrr::map(clean_con, scrape_individual_article)    # Using our pre-made f
bind_rows(details_kw3)

# Turning the binded rows into a tibble
details_kw3 <- bind_rows(details_kw3)

```

KEYWORD 4: “Artificial Intelligence” 2 PAGES OF WEBSITE

```

# Scraping links from PAGE 1 - ARTIFICIAL INTELLIGENCE
pg1_ai <- scrape_list_links1("https://link.springer.com/search?query=artificial+intelligence")

# Scraping links from PAGE 2 - ARTIFICIAL INTELLIGENCE
pg2_ai <- scrape_list_links1("https://link.springer.com/search?query=artificial+intelligence")

# Combining two pages into one list of links (38 links total)
ai <- list(pg1_ai, pg2_ai)
clean_ai <- unlist(ai)

```

```

# Scraping information from each individual article.
details_kw5 <- purrr::map(clean_ai, scrape_individual_article)      # Using our pre-made f
bind_rows(details_kw5)

# Turning the binded rows into a tibble
details_kw5 <- bind_rows(details_kw5)

# Combining all data tables with different key words into one tibble
all_articles <- bind_rows(
  details_kw1 |>
    mutate(keyword = "machine learning"),
  details_kw2 |>
    mutate(keyword = "deep learning"),
  details_kw3 |>
    mutate(keyword = "convolution"),
  details_kw5 |>
    mutate(keyword = "artificial intelligence")
)

# Creating the CSV file
write_csv(all_articles, "springer_articles.csv")

```